

THE-772 · B2B 报价系统 — 上手学习教程

Hands-on Learning Guide

目的：跟着这份教程一步步操作，**对照 THE-772 ticket 的每一条要求**，理解这套系统里每个功能「**是干嘛的、怎么用、期望看到什么结果、背后怎么实现**」。读法：边读边在浏览器里点（建议把本教程和 `http://localhost:3000` 并排放）。每一课结构固定： 这一课要懂什么 ·  对应 ticket 哪条 ·  动手做 ·  期望结果 ·  背后原理 ·  小结。

准备：先把系统跑起来 / Setup

```
npm install --ignore-scripts          # 装依赖（跳过原生编译）
# 取 better-sqlite3 预编译二进制（Node 24 = ABI v137）：
node -e "const fs=require('fs');fetch('https://registry.npmmirror.com/-/binary/better-sqlite3-0.10.2.tar.gz').then(r=>r.blob().then(b=>fs.writeFileSync('bsq.tgz',b)))"
tar -xzf bsq.tgz -C node_modules/better-sqlite3 && rm bsq.tgz
npm run dev                          # 打开 http://localhost:3000
```

打开后你会看到左侧有一条深色侧边栏，分成两组——这就是理解整套系统的第一把钥匙：

- **Retailer Portal（零售商门户）**：Dashboard / Catalog / Quotes / Pre-Orders —— 这是**零售商**（你的客户，比如室内设计公司）用的界面。
- **Supply Chain（供应链）**：Supplier Console / Pricing Versions —— 这是**系统后台 / 供应商侧**，零售商看不到。

 **一句话定位系统**：零售商在门户里「选产品 → 配尺寸 → 看渲染 → 自动出价 → 下预订单」，系统把订单变成供应商能用的 Excel，供应商回传订单号和物流单号，状态再实时回到零售商眼前——一条从「报价」到「送货上门」的完整链路。

📖 **数据怎么流的**（记住这张图，后面每一课都在这条链上）：

目录（静态数据）→ 配置器 → 渲染 + 后端报价引擎 → 报价草稿 → 预订单

↓

供应商 Excel ← ← ← ← ← ← ← ← ←

↓

Supplier Console（模拟供应商+物流）→ 订单号/物流号/状态

↓

事件 order_events → 零售商时间线 + Dashboard 动态流

模块一：零售商前端 UX

Module 1 — Retailer-facing UX

对应 ticket 「System overview §1. Retailer-facing UX」全部 5 条要求。

第 1 课：Dashboard —— 建立全局心智模型

- 🧠 **要懂什么**：先不深入细节，建立「这个账户在发生什么」的整体感。
- 📄 **对应 ticket**：retailer-facing 门户的总览入口。
- 🛠 **动手做**：打开 🏠（首页）。从上往下看：
 - 顶部四个数字卡：Open quotes（未结报价）、Active pre-orders（在途预订单）、In production（生产中）、Delivered（已妥投）。
 - 中间两张大图：Roller Shade / Drapery 两条产品线入口。
 - 左下 **Fulfillment pipeline**：每个订单状态各有几单。
 - 右下 **Latest updates**：供应商/物流推来的最新动态流。
- ✅ **期望结果**：你会看到系统**自带 2 条历史订单**（演示数据）——一条已 Delivered、一条 In Production。动态流里能看到「Supplier 确认订单」「物流妥投」之类的记录。这说明数据库已自动建好并灌入了种子数据。

- 🔍 **背后原理**: `app/page.tsx` 是服务端组件, 直接调 `lib/db.ts` 的 `getOrders/getQuotes/getRecentEvents` 读 SQLite。数据库在首次请求时自动建表+灌种子 (`lib/db.ts` 的 `seed()`) 。
- 💡 **小结**: Dashboard = 零售商的「驾驶舱」。后面你做的每一步 (下单、推进状态) 都会回流到这里。

第 2 课: Catalog —— 策展目录 + 「变体约束是数据」

- 🎯 **要懂什么**: 目录不是随便堆产品, 而是只放供应链能生产的; 而且每个花色只开放部分遮光度。
- 📋 **对应 ticket**: 「Curated catalog of supply-chain-producible products」+ 「Variations/combinations: pattern + opacity options **valid for that pattern**」。
- 🛠 **动手做**:
 1. 打开 `/catalog`。顶部可按产品线 (Roller Shade / Drapery) 筛选。
 2. 看每张产品卡底部的灰色小标签——那是这个花色「可选的遮光度」。
 3. 对比两个产品: `Solar Screen 3%` (只有 **Sheer** 一个标签) vs `Aria Solid` (有 Light Filtering / Room Darkening / Blackout) 。
- ✅ **期望结果**: 你会发现不同花色开放的遮光度不一样。这就是「变体约束」——不是所有「花色 × 遮光度」组合都能造。
- 🔍 **背后原理**: 目录是静态 TypeScript 数据 (`lib/catalog-data.ts`), 不是数据库行。每个产品有个 `validOpacities` 字段, 比如 `rs-solar` 是 `["sheer"]`、`rs-midnight` 是 `["blackout"]`。卡片上的标签就是直接渲染这个字段。
- 💡 **小结**: 「能不能造」是数据决定的, 不是写在代码里的 if-else。这让后面加新产品/新约束只需要改数据。

第 3 课: 配置器 (核心页) —— 一次搞懂 4 件事

配置器是整个系统的核心, ticket \$1 里有 4 条要求都在这一页。我们拆成 4 个小步。打开 `/configure/rs-botanica` (Botanica, 一个设计师款卷帘)。

3a · 变体约束的「实时拦截」

- 🎯 **要懂什么**: 约束不只展示, 还会阻止你选不能造的组合。
- 📋 **对应 ticket**: variations valid for that pattern (前端 + 后端双重保证)。
- 🛠 **动手做**: 在右侧 **Opacity** 区, 注意有些遮光度是置灰、带删除线、点不动的。换一个产品试: 打开 `/configure/rs-solar`, 你会发现除了 Sheer 全是灰的。
- ✅ **期望结果**: 不可生产的遮光度无法选中, 鼠标悬停提示「Not producible for this pattern」。

- 🔍 **背后原理**：前端读 `product.validOpacities` 把非法项 disable (`components/Configurator.tsx`)；**更重要的是后端也会校验**——`lib/pricing.ts` 的 `validateConfig` 在 `POST /api/price` 里再查一遍，非法组合直接返回 HTTP 422。前端可以被绕过，后端不行。
- 💡 **小结**：「客户端做体验，服务端做真相」——这是贯穿全系统的安全设计。

3b · 在场景中的实时渲染

- 🎯 **要懂什么**：每改一个参数，左边的产品效果图**立刻重画**。
- 📋 **对应 ticket**：「In-context render generation for the selected configuration」。
- 🛠️ **动手做**：在右侧改颜色、改遮光度、改尺寸 (Width/Height)、改选项 (Mount/Headrail/Control)；再拖动渲染图下方的**滑杆**。
- ✅ **期望结果**：左侧的窗户场景图会随每次改动即时更新——颜色变了、卷帘升降位置变了、cassette/开放轴的样子变了。
- 🔍 **背后原理**：`components/renderers.tsx` 是**参数化 SVG**，把配置当参数画图。⚠️ **注意**：这是**程序化 SVG 占位**，代表「真实可视化引擎的接入点」，不是最终渲染产品（见 ticket 的 render integration 议题）。
- 💡 **小结**：渲染在架构上是「一个会随配置实时更新的接口」，未来换成真引擎，接口不变。

3c · 参数化的尺寸输入

- 🎯 **要懂什么**：不同产品线问的尺寸**不一样**，而且有范围校验。
- 📋 **对应 ticket**：「Dimension input: drapery fabric height/width, rod width, etc. — **parameterized per product line**」。
- 🛠️ **动手做**：
 1. 卷帘页 (`/configure/rs-botanica`) 问的是 **Width / Height**。
 2. 打开一个窗帘 `/configure/dp-velluto`，问的变成 **Rod / track width / Finished height**——字段不同！
 3. 在卷帘页把 Width 填成 `9999`（超范围）。
- ✅ **期望结果**：超范围时输入框变红、下方提示有效区间（卷帘宽 30–300cm），并且**报价会暂停**（先改对再算）。
- 🔍 **背后原理**：每条产品线在 `lib/catalog-data.ts` 里定义自己的 `dimensionFields` (key/label/min/max/step)。配置器据此**动态生成**输入框；后端 `validateConfig` 再校验范围。
- 💡 **小结**：加一条新产品线、问完全不同的尺寸，只是加一份数据，不用改配置器代码。

3d · 后端公式自动报价

- 🎯 **要懂什么**：价格是**后端公式实时算的**，还给出每一项的明细。
- 📋 **对应 ticket**：「Auto-quote calculated by backend formulas from catalog + variation + dimensions」。

- 🛠 **动手做：**在 `/configure/rs-midnight` (Midnight 卷帘, 仅 Blackout) 里: 选 Blackout、Control 选 **Motorized**、尺寸填 Width `120` / Height `160`。看右下角单价 + 左下「Quote breakdown」明细。
- ✅ **期望结果：**单价应为 **\$196.40**。明细会拆出: 尺寸分档基价 → 遮光/档位系数 → 各选项加价 (电动 +\$95)。价格卡右上角标着 **pricing v2026.2**。
- 🔍 **背后原理：**配置器把配置防抖 (280ms) 发给 `POST /api/price`; `lib/pricing.ts` 用价目表/公式算出 `unitPrice + 明细 lines + 工艺参数 facts`。卷帘走「尺寸分档价目表」(roller-grid), 窗帘走「用料公式」(drapery-formula, 见第 9 课)。
- 💡 **小结：**你看到的每个价格都来自一个**纯函数引擎**, 可解释、可审计。试试换个窗帘 (如 `dp-eclipse`) , 明细会变成「用布米数 × 单价 + 缝制 + 衬里」这种制造维度。

模块二：预订单 + 供应商履约管线

Module 2 — Pre-order & supplier fulfillment pipeline

对应 ticket 「System overview §2」全部要求。

第 4 课：加入报价 & Quotes —— 草稿在累积，价格在服务端重算

- 🎯 **要懂什么：**配置好的产品先进「草稿报价单」, 可以攒多条; 而且**入库价格由服务端重新计算**。
- 📄 **对应 ticket:** retailer confirms the quote (报价是预订单的前一步) 。
- 🛠 **动手做：**
 1. 在任意配置页点 **Add to quote** (可调数量 Qty) 。
 2. 左侧栏 Quotes 旁出现数字角标; 打开 `/quotes` 看草稿, 点进去看汇总。
- ✅ **期望结果：**草稿里累积了你加的明细; 汇总显示行数、件数、**锁定的定价版本**、总价 (FOB) 。
- 🔍 **背后原理：**`POST /api/quote-items` 收到请求后**不信任前端传来的价格**, 用 `computeQuote` **重新算一遍**再入库 (`lib/db.ts` 的 `addQuoteItem`)。每条报价行存的是「完整配置 + 计算结果 + 版本号」三件套的快照。
- 💡 **小结：**报价行是「自包含快照」——这是后面「调价不影响历史报价」的基础。

第 5 课：提交预订单 —— 报价转订单

- 🎯 **要懂什么：**确认报价后一键变成「预订单」, 进入履约管线。

- 📄 **对应 ticket**: 「Retailer confirms the quote → submits a pre-order」。
- 🛠 **动手做**: 在草稿报价详情页点 **Submit pre-order** →, 会有二次确认, 点 Confirm。
- ✅ **期望结果**: 跳转到新建的预订单详情页 (编号形如 `PO-2026-00xx`) ; 状态停在第一站 **Submitted**; 报价单状态变成 **Converted**。
- 🔍 **背后原理**: `POST /api/quotes/:id/submit` → `submitPreOrder` (`lib/db.ts`) 在一个事务里: 把报价标为 converted、建 order、写第一条 `order_events` ("Pre-order 已提交, 供应商订单文件已生成并排队投递")。
- 💡 **小结**: 从这一步起, 「报价」结束, 「履约」开始。

第 6 课: 供应商 Excel —— 系统说供应商的语言

- 🎯 **要懂什么**: 系统能把订单生成**供应商能直接用的中英双语 Excel**。
- 📄 **对应 ticket**: 「System generates an Excel order file in the format the China supplier expects」。
- 🛠 **动手做**: 在预订单详情页点右上角 ↓ **Supplier order file (.xlsx)**, 下载并用 Excel 打开。
- ✅ **期望结果**: 一个工作簿, 含: 抬头块 (订单号/报价号/零售商/项目/日期)、**每行明细** (产品线、SKU、花色、遮光、宽高、选项、**工艺参数**如用布米数和幅数、数量、单价)、合计、外加一页中英双语「说明 Instructions」。表头都是「中文 / English」双语。
- 🔍 **背后原理**: `lib/excel.ts` 用 exceljs 构建工作簿, `GET /api/orders/:id/excel` 提供下载。每行的工艺参数直接取自报价行快照里的 `computation.facts`。
- 💡 **小结**: 这是「我们系统」和「供应商系统」之间的**数据契约**。⚠ 目前是**下载**, 真实投递 (邮件/SFTP/API) 是后续阶段 (见 ticket 的 supplier handoff 议题)。

第 7 课: 订单详情 —— 状态机 + 时间线

- 🎯 **要懂什么**: 订单有一条**固定的状态流水线**, 每次推进都会留痕。
- 📄 **对应 ticket**: 「ongoing fulfillment status updates ... Real-time status pushed back to the retailer until final delivery」。
- 🛠 **动手做**: 看任意预订单详情页 (比如种子数据里的 `PO-2026-0002`)。看顶部 6 段**状态步进条**、右侧履约信息 (供应商单号/运单号/承运商/ETA)、底部 **Timeline 时间线**。
- ✅ **期望结果**: 步进条 `Submitted → Acknowledged → In Production → Shipped → In Transit → Delivered`, 当前站点亮; 时间线按时间倒序列出每一次状态变更, 标注是谁推的 (You / Supplier / Logistics)。
- 🔍 **背后原理**: 状态枚举 `ORDER_STATUSES` (`lib/types.ts`)。每一次状态变更都写一条 `order_events` ——时间线、Dashboard 动态流读的都是这张表。事件就是「给零售商的更新通道」。
- 💡 **小结**: 「状态」是结果, 「事件」是过程记录。零售商看到的一切更新, 本质都是 `order_events` 行。

第 8 课: Supplier Console —— 模拟供应商+物流, 亲手推进全链路

- 🌀 **要懂什么:** 站在供应商/物流的角度, **亲手推进**一个订单走完全程, 看更新如何实时回到零售商侧。
- 📄 **对应 ticket:** 「Supplier returns an Order Number + status updates」 「Order № + Tracking № sync into logistics」 「status pushed back to the retailer」。
- 🛠 **动手做:**
 1. 打开 `/supplier` (Supplier Console)。它有个黄条提示「这是模拟面」。
 2. 找到你第 5 课新建的订单, 点最右的动作按钮, 依次推进: **Acknowledge (发订单号)** → **Start production** → **Ship (发运单号)** → **Mark in transit** → **Mark delivered**。
 3. 每点一次, 回到零售商侧 `/orders/<id>` 或 Dashboard 看变化。
- ✅ **期望结果:**
 - Acknowledge 后订单出现一个 **Supplier №** (形如 `SZF-12345`) 和 ETA;
 - Ship 后出现 **Tracking №** (形如 `SF ...`) 和承运商;
 - 每一步都会在零售商侧的时间线和 Dashboard 动态流里**即时多出一条记录**;
 - 走完后按钮变成「Complete ✓」。
 - 试试乱序 (比如还没 Acknowledge 就想 Ship) ——做不到, 后端会拦。
- 🔍 **背后原理:** `POST /api/orders/:id/advance` (`app/api/orders/[id]/advance/route.ts`)。它有一张**前置条件表**: 必须 `submitted` 才能 acknowledge、必须 `in_production` 才能 ship.....乱序返回 HTTP 409。订单号/运单号在对应动作里生成。⚠ 这些号目前是**随机生成、承运商写死的**——真实集成 (供应商回调、承运商 API) 是后续阶段。
- 💡 **小结:** 这个面把「本该由外部系统发来的事件」用按钮模拟出来, 让你能端到端跑通。它**代表真实集成**的接入点。

模块三: 定价引擎的版本控制

Module 3 — Pricing version control

对应 ticket 架构议题 「Quote formula engine — formula configurability, **version control of pricing**」。

第 9 课: Pricing Versions —— 调价不动历史报价

- 🌀 **要懂什么**: 报价公式是**可配置、可版本化**的; 老报价永远按当时的版本算, 调价不会篡改历史。
 - 📄 **对应 ticket**: 「formula configurability, version control of pricing」。
 - 🛠 **动手做**:
 1. 打开 `/pricing`。看 Roller Shade 下有两个版本: **v2026.1 (Superseded 已停用)** 和 **v2026.2 (Active 现行)**。
 2. 对比两个版本卡上的小标签: v2026.1 是 **Motorized +\$90 / Blackout × 1.28**, v2026.2 是 **Motorized +\$95 / Blackout × 1.30**。
 3. 看 Drapery 那一版——展示的是一条**用料公式** ($\text{widths} = \text{ceil}(\text{rodWidth} \times \text{fullness} \div \text{panels} \div 140\text{cm})$...)。
 - ✅ **期望结果**: 两个卷帘版本数值确实不同 (这正是第 3d 课你算出 \$196.40 用的是现行 v2026.2)。今天新建的报价锁定 v2026.2; 而种子里的历史报价若用旧版, 则永远按旧版价显示。
 - 🔍 **背后原理**: 定价配置是 `pricing_versions` 表里的**版本化行** (`lib/db.ts`) ; 报价行快照里存了 `pricingVersion`。`lib/pricing.ts` 是纯函数, 给它哪个版本的配置就按哪个算。卷帘=价目表 `roller-grid`, 窗帘=用料公式 `drapery-formula`。
 - 💡 **小结**: 「价格可演进, 但每张报价都钉死在它出生时的价目表上」——这就是 B2B 报价系统能被信任的关键。
-

第 10 课: 回看全链路 + 哪些是真的、哪些是模拟

你已经走完一整条链。现在把它和 ticket 的范围对齐一次:

环节	ticket 要求	这套系统
策展目录 + 变体约束	\$1	✔ 真实 (数据驱动)
实时渲染	\$1	● 程序化 SVG 占位
参数化尺寸 + 自动报价	\$1	✔ 真实 (后端公式引擎)
报价 → 预订单	\$2	✔ 真实
供应商 Excel 生成	\$2	✔ 真实 (中英双语)
Excel 投递 / 供应商回传 / 物流单号 / 推送给零售商	\$2	● 由 Supplier Console 人工模拟
定价版本控制	架构议题	✔ 真实
仅 Roller + Drapery	\$3	✔ 符合

- 🌀 **最终心智模型**：左半条链（目录→配置→报价→预订单→Excel生成）是**真实可用**的；右半条链（Excel投递→供应商回传→物流→实时推送）目前是**有意模拟**的占位——这正是 ticket 划定的范围（实现留待后续阶段）。
- 💡 **你现在应该能**：(1) 解释系统每个页面是干嘛的；(2) 端到端跑通一单；(3) 说清哪些是真实实现、哪些是模拟接入点；(4) 对照 ticket 指出系统覆盖了哪些要求。

想再深一层（看代码怎么搭的），读仓库里的 `CLAUDE.md`（架构总览）和 `THE-772-PROTOTYPE-GUIDE.md`（逐条对照 + 缺口分析）。